HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI TẬP
HỌC PHẦN: IOT VÀ ỨNG DỤNG
MÃ HỌC PHẦN: INT14149

ĐỀ TÀI: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG IOT GIÁM SÁT NHIỆT ĐỘ, ĐỘ ẨM, ÁNH SÁNG VÀ ĐIỀU KHIỂN ĐÈN LED TỪ XA

Sinh viên thực hiện:

B23DCAT162 Văn Đức Kiên

Giảng viên hướng dẫn: TS. Nguyễn Quốc Uy

HÀ NỘI 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành tới TS. Nguyễn Quốc Uy, giảng viên học phần "IoT và Ứng dụng", đã giảng dạy và hướng dẫn em trong quá trình học tập cũng như thực hiện bài tập lớn.

Những kiến thức và góp ý của thầy đã giúp em hiểu rõ hơn cách xây dựng một hệ thống IoT, từ kết nối phần cứng, truyền nhận dữ liệu đến phát triển ứng dụng giám sát và điều khiển. Đây là cơ sở để em vận dụng vào đề tài và hoàn thiện bài báo cáo.

Em kính chúc thầy luôn mạnh khỏe, hạnh phúc và đạt nhiều thành công trong công tác giảng dạy, nghiên cứu.

Em xin chân thành cảm ơn!

Hà Nội, ngày 15 tháng 10 năm 2026

MỤC LỤC

LỜI CẢM ƠN 2
MỤC LỤC 3
DANH MỤC CÁC HÌNH VẼ 5
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI 6
1.1 Giới thiệu đề tài 6
1.2 Lý do chọn đề tài 6
1.3 Mục tiêu của đề tài 7
1.4 Đối tượng và phạm vi thực hiện 7
1.5 Ý nghĩa của đề tài 8
CHƯƠNG 2. THIẾT KẾ HỆ THỐNG 9
2.1 Mô hình hoạt động 9
2.2 Thiết kế chức năng hệ thống 10
2.2.1 Usecase login 11
2.2.2 Usecase Dashboard & Realtime Chart 13
2.2.3 Usecase Sensor Data 15
2.2.4 Usecase Control Led 17
2.2.5 Usecase Action History 22
2.2.6 Usecase Profile 23
CHƯƠNG 3. XÂY DỰNG HỆ THỐNG 26
3.1 Xây dựng phần cứng 26
3.1.1 Các thành phần phần cứng 26
3.1.2 Ngôn ngữ và thư viện sử dụng 27
3.1.3 Hoạt động của phần cứng 27
3.2 Frontend 28
3.2.1 HTML, CSS và JavaScript 28
3.2.2 Fetch API 28
3.2.3 Chart.js 28
3.2.4 Socket.IO Client 29
3.3 Backend 29
3.3.1 JavaScript và Node.js 29
3.3.2 Express 29
3.3.3 MQTT.js và Mosquitto 29
3.3.4 Socket.IO 30

3.4 Database 30
3.4.1 Bảng users 31
3.4.2 Bảng devices 31
3.4.3 Bảng sensors 32
3.4.4 Bảng data_sensor 32
3.4.5 Bảng action_history 32
3.5 API 33
CHƯƠNG 4. KẾT QUẢ THỰC HIỆN 34
4.1 Giao diện login 34
4.2 Giao diện dashboard 34
4.3 Giao diện Data Sensor 35
4.4 Giao diện Control Led 35
4.5 Giao diện Action History 36
4.6 Giao diện Profile 37
KẾT LUẬN 38
TÀI LIỆU THAM KHẢO 39

DANH MỤC CÁC HÌNH VẼ

Hình 1 Sơ đồ tổng quan....10
Hình 2 Sơ đồ usecase hệ thống....11
Hình 3 Biểu đồ sequence usecase login....13
Hình 4 Biểu đồ sequence usecase Usecase Dashboard & Realtime Chart....15
Hình 5 Biểu đồ sequence usecase sensor data....17
Hình 6 Biểu đồ sequence use case control led....21
Hình 7 Biểu đồ sequence usecase Action History....23
Hình 8 Biểu đồ sequence usecase profile....25
Hình 9 Thiết kế mạch....27
Hình 10 Các bảng CSDL....31
Hình 11 Giao diện login....34
Hình 12 Giao diện dashboard....34
Hình 13 Giao diện Data Sensor....35
Hình 14 Giao diện Control led....36
Hình 15 Giao diện Action History....36
Hình 16 Giao diện Profile....37

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu đề tài

Internet vạn vật (Internet of Things – IoT) cho phép kết nối các thiết bị vật lý thông qua mạng truyền thông nhằm thu thập, trao đổi dữ liệu và thực hiện các hoạt động giám sát, điều khiển. Trong một hệ thống IoT, cảm biến ghi nhận thông tin từ môi trường, bộ điều khiển tiếp nhận và truyền dữ liệu, còn ứng dụng phần mềm cung cấp giao diện để người dùng theo dõi và tương tác với thiết bị.

Nhiệt độ, độ ẩm và ánh sáng là những thông số phản ánh điều kiện môi trường, có liên quan đến sinh hoạt, học tập, làm việc và sản xuất. Việc theo dõi các thông số này giúp Người dùng nắm bắt tình trạng môi trường tại khu vực quan sát. Khi dữ liệu được lưu trữ, người dùng còn có thể xem lại các lần đo và nhận biết sự thay đổi theo thời gian.

Bên cạnh chức năng giám sát, điều khiển thiết bị từ xa giúp người dùng thực hiện thao tác mà không cần trực tiếp đến vị trí lắp đặt. Việc kết hợp giám sát môi trường và điều khiển thiết bị trên cùng một giao diện tạo nên mô hình quản lý tập trung, thuận tiện cho việc quan sát và vận hành.

Xuất phát từ nhu cầu đó, đề tài "Thiết kế và xây dựng hệ thống IoT giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển đèn LED từ xa" được thực hiện nhằm xây dựng một mô hình kết hợp giữa phần cứng và ứng dụng web. Hệ thống sử dụng ESP32 để thu thập dữ liệu cảm biến và điều khiển ba đèn LED. Dữ liệu được truyền về máy chủ để xử lý, lưu trữ và hiển thị trên giao diện. Người dùng có thể theo dõi các thông số mới nhất, quan sát biểu đồ, tra cứu lịch sử và gửi yêu cầu bật hoặc tắt từng đèn.

1.2 Lý do chọn đề tài

Trong phương pháp giám sát trực tiếp, Người dùng cần có mặt tại vị trí đặt thiết bị để đọc thông số hoặc thực hiện thao tác điều khiển. Cách thức này gây bất tiện khi cần theo dõi thường xuyên hoặc khi Người dùng ở xa khu vực lắp đặt. Nếu dữ liệu không được lưu trữ, việc xem lại điều kiện môi trường và đối chiếu các thời điểm khác nhau cũng gặp khó khăn.

Ứng dụng IoT cho phép đưa dữ liệu từ cảm biến lên một hệ thống phần mềm tập trung. Thông qua giao diện web, người dùng có thể theo dõi thông số, quan sát biểu đồ và tra cứu dữ liệu đã ghi nhận. Các yêu cầu điều khiển được truyền qua mạng, giúp người dùng tương tác với thiết bị từ một vị trí khác khi có kết nối phù hợp đến máy chủ.

Đề tài được lựa chọn vì phù hợp với việc vận dụng kiến thức về lập trình nhúng, truyền thông mạng, cơ sở dữ liệu và phát triển ứng dụng web. Quá trình thực hiện không chỉ dừng lại ở việc đọc dữ liệu cảm biến hay bật, tắt đèn, mà còn đặt ra yêu cầu phối hợp giữa các thành phần: thiết bị, hạ tầng truyền thông, máy chủ và giao diện người dùng.

Việc sử dụng ESP32, cảm biến nhiệt độ và độ ẩm, cảm biến ánh sáng cùng đèn LED tạo điều kiện xây dựng mô hình thử nghiệm có quy mô phù hợp. Ba đèn LED đóng vai trò thiết bị chấp hành, giúp minh họa quá trình gửi lệnh, thực hiện thao tác và phản hồi kết quả. Đây cũng là cơ sở để nghiên cứu cách xử lý các tình huống như mất kết nối, thiết bị không phản hồi hoặc yêu cầu điều khiển hết thời gian chờ.

1.3 Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là thiết kế và xây dựng một hệ thống IoT có khả năng giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển đèn LED từ xa thông qua giao diện web. Hệ thống cần thực hiện được quá trình thu thập dữ liệu, truyền thông, xử lý, lưu trữ và cung cấp thông tin cho người dùng.

Các mục tiêu cụ thể bao gồm:

- Xây dựng mô hình phần cứng sử dụng ESP32, cảm biến DHT11, cảm biến ánh sáng LDR và ba đèn LED.

- Thu thập dữ liệu cảm biến theo chu kỳ và truyền dữ liệu về máy chủ thông qua mạng Wi-Fi.

- Thiết lập quá trình trao đổi dữ liệu cảm biến, lệnh điều khiển và phản hồi giữa ESP32 với máy chủ.

- Xây dựng giao diện web hiển thị các thông số môi trường mới nhất, biểu đồ và trạng thái kết nối của thiết bị.

- Cho phép người dùng bật hoặc tắt độc lập từng đèn LED và theo dõi kết quả xử lý lệnh.

- Lưu trữ dữ liệu cảm biến, trạng thái thiết bị và lịch sử điều khiển để phục vụ tra cứu.

- Cung cấp chức năng tìm kiếm, lọc, sắp xếp và phân trang đối với dữ liệu lịch sử.

- Hỗ trợ đăng ký, đăng nhập và xác thực người dùng khi truy cập các chức năng của hệ thống.

Ngoài các chức năng trên, đề tài hướng đến việc phân biệt rõ giữa yêu cầu đã được máy chủ tiếp nhận và lệnh đã được thiết bị thực hiện thành công. Kết quả điều khiển cần được xác định thông qua phản hồi từ ESP32, giúp người dùng theo dõi đúng tình trạng xử lý và nhận biết các trường hợp thất bại.

1.4 Đối tượng và phạm vi thực hiện

Đối tượng nghiên cứu của đề tài là mô hình giám sát môi trường và điều khiển thiết bị thông qua mạng, trong đó tập trung vào sự phối hợp giữa ESP32, cảm biến, đèn LED và ứng dụng web. Các nội dung được tìm hiểu gồm thu thập dữ liệu, truyền thông giữa thiết bị với máy chủ, lưu trữ thông tin và hiển thị kết quả cho Người dùng.

Về phần cứng, đề tài triển khai một nút ESP32 kết nối với cảm biến DHT11 để đo nhiệt độ và độ ẩm, cảm biến LDR để ghi nhận mức độ ánh sáng và ba đèn LED để mô phỏng các thiết bị cần điều khiển. Các đèn được bật hoặc tắt độc lập thông qua yêu cầu từ giao diện web.

Về phần mềm, hệ thống gồm máy chủ xử lý, cơ sở dữ liệu, thành phần trung gian truyền bản tin và giao diện web. Các chức năng được triển khai tập trung vào đăng ký, đăng nhập,

giám sát dữ liệu môi trường, hiển thị biểu đồ, điều khiển LED, xem trạng thái kết nối và tra cứu lịch sử.

Về môi trường triển khai, hệ thống được xây dựng và kiểm thử chủ yếu trong mạng nội bộ. Khả năng điều khiển từ xa được thực hiện khi trình duyệt và thiết bị có kết nối phù hợp đến máy chủ. Việc truy cập qua Internet cần có thêm phương án triển khai máy chủ và cấu hình mạng tương ứng.

Trong phạm vi đề tài, các thao tác điều khiển được thực hiện thủ công bởi người dùng. Hệ thống chưa tập trung vào tự động điều khiển theo ngưỡng cảm biến, gửi cảnh báo qua tin nhắn, quản lý nhiều nút ESP32 hoặc điều khiển thiết bị điện công suất lớn. Các nội dung này được xem xét trong hướng phát triển của đề tài.

1.5 Ý nghĩa của đề tài

Về mặt học tập, đề tài giúp vận dụng tổng hợp kiến thức về phần cứng, lập trình vi điều khiển, truyền thông mạng, cơ sở dữ liệu và xây dựng ứng dụng web. Thông qua việc triển khai một mô hình hoàn chỉnh, người thực hiện có thể hiểu rõ hơn quá trình dữ liệu được hình thành tại cảm biến, truyền về máy chủ, lưu trữ và hiển thị trên giao diện.

Quá trình thực hiện còn giúp rèn luyện kỹ năng phân tích yêu cầu, thiết kế hệ thống, tổ chức mã nguồn và kiểm thử. Việc xử lý các tình huống như mất kết nối, lệnh chưa được xác nhận hoặc dữ liệu chưa được cập nhật giúp làm rõ những vấn đề cần quan tâm khi xây dựng ứng dụng có tương tác với thiết bị vật lý.

Về mặt thực tiễn, đề tài minh họa khả năng xây dựng hệ thống giám sát và điều khiển tập trung bằng các linh kiện phổ biến. Giao diện web hỗ trợ người dùng theo dõi thông số và thực hiện thao tác trên cùng một nền tảng, trong khi dữ liệu lịch sử tạo điều kiện xem lại các lần đo và hoạt động điều khiển.

Mô hình có thể được sử dụng làm nền tảng cho các ứng dụng giám sát phòng học, phòng làm việc hoặc khu vực trồng cây. Khi mở rộng sang môi trường sử dụng thực tế, hệ thống cần được bổ sung và đánh giá về độ chính xác của cảm biến, độ ổn định kết nối, khả năng bảo vệ truy cập và phần cứng chấp hành phù hợp.

CHƯƠNG 2. THIẾT KẾ HỆ THỐNG

2.1 Mô hình hoạt động

Luồng giám sát dữ liệu

Hardware → MQTT Broker → Backend Server → Database.

Backend Server → Frontend/Dashboard → Người dùng.

Hardware thu thập dữ liệu nhiệt độ, độ ẩm và ánh sáng từ các cảm biến theo chu kỳ, sau đó gửi đến MQTT Broker. Backend Server tiếp nhận bản tin, kiểm tra dữ liệu và lưu vào cơ sở dữ liệu. Khi có dữ liệu mới, Backend thông báo cho Frontend qua Socket.IO. Frontend gọi API để cập nhật các thông số và biểu đồ trên Dashboard.

Khi mở trang hoặc tải lại giao diện, Frontend lấy dữ liệu đã lưu thông qua Backend, không truy cập trực tiếp vào Database.

Luồng điều khiển LED

Người dùng → Frontend/Dashboard → Backend Server → MQTT Broker → Hardware.

Người dùng thực hiện thao tác bật hoặc tắt LED trên Dashboard. Frontend gửi yêu cầu đến Backend thông qua API, kèm thông tin xác thực. Backend kiểm tra yêu cầu, lưu lệnh vào cơ sở dữ liệu với trạng thái chờ xử lý (PENDING), sau đó gửi lệnh qua MQTT Broker. Hardware nhận lệnh và điều khiển LED tương ứng.

Lệnh được tiếp nhận chưa đồng nghĩa với việc điều khiển đã thành công. Kết quả thực hiện cần được xác nhận bằng phản hồi từ Hardware.

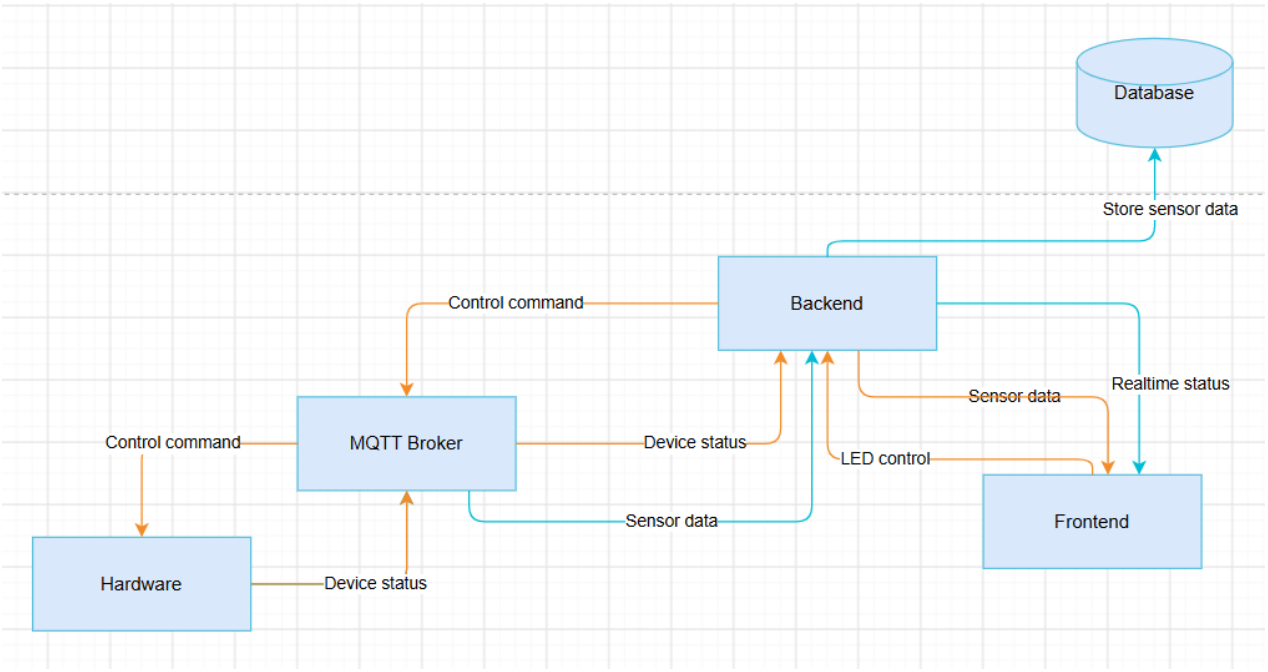
Luồng phản hồi kết quả điều khiển

Hardware → MQTT Broker → Backend Server → Database.

Backend Server → Frontend/Dashboard → Người dùng.

Sau khi xử lý lệnh, Hardware gửi phản hồi chứa mã lệnh, ID thiết bị, trạng thái LED và kết quả thực hiện qua MQTT Broker. Backend đối chiếu phản hồi với lệnh đang chờ, cập nhật kết quả vào cơ sở dữ liệu và cập nhật trạng thái LED nếu lệnh thành công. Kết quả được gửi đến Frontend qua Socket.IO để hiển thị cho người dùng. Nếu không nhận được phản hồi hợp lệ trong thời gian quy định, lệnh được chuyển sang trạng thái thất bại (FAILED).

Các bản tin thông báo trạng thái kết nối và trạng thái LED được xử lý riêng, không thay thế phản hồi xác nhận kết quả của từng lệnh.



Hình 1 Sơ đồ tổng quan

2.2 Thiết kế chức năng hệ thống

Hệ thống được phân chia thành sáu Use Case chính, tương ứng với các chức năng mà người dùng thực hiện trên giao diện web:

- Login: Xác thực tài khoản để truy cập hệ thống.

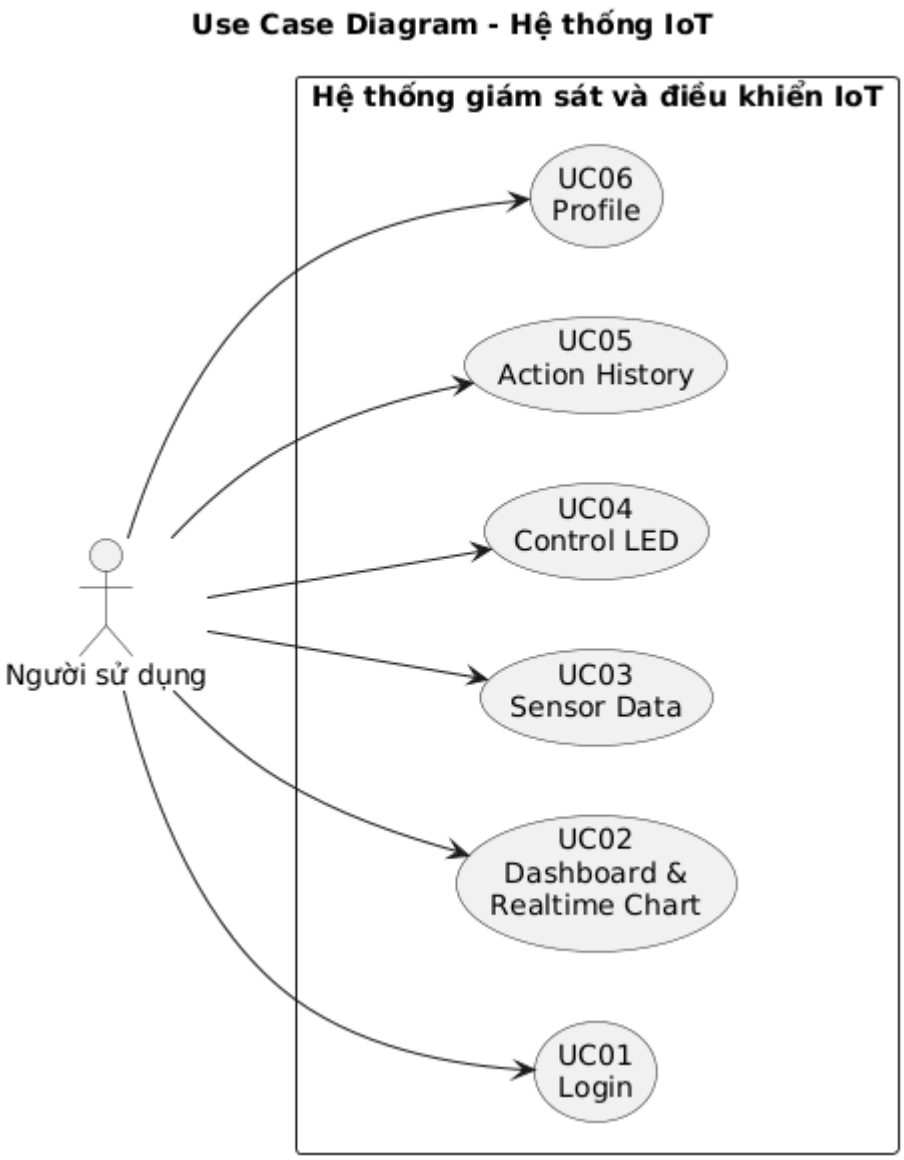
- Dashboard & Realtime Chart: Theo dõi các thông số môi trường, trạng thái thiết bị và biểu đồ cập nhật theo thời gian thực.

- Sensor Data: Xem, tìm kiếm, lọc và tra cứu dữ liệu cảm biến đã lưu.

- Control LED: Gửi yêu cầu bật hoặc tắt từng đèn LED và theo dõi kết quả thực hiện.

- Action History: Tra cứu lịch sử các lệnh điều khiển LED.

- Profile: Xem thông tin cá nhân và truy cập các liên kết tài liệu liên quan.



Hình 2 Sơ đồ usecase hệ thống

2.2.1 Usecase login

2.2.1.1 Mô tả usecase

- Cho phép Người dùng xác thực tài khoản để truy cập vào hệ thống.
- Tiếp nhận thông tin đăng nhập, kiểm tra tài khoản và mật khẩu. Nếu thông tin hợp lệ, hệ thống cho phép Người dùng truy cập Dashboard; nếu không hợp lệ, hệ thống trả về thông báo lỗi.

Actor: Người dùng

Tiền điều kiện: Người dùng đã có tài khoản và hệ thống đang hoạt động.

Hậu điều kiện: Đăng nhập thành công, Người dùng được truy cập vào Dashboard.

Luồng chính:

1. Người dùng truy cập trang Login.

2. Người dùng nhập username và password.
3. Người dùng chọn chức năng đăng nhập.
4. Hệ thống kiểm tra thông tin đăng nhập.
5. Hệ thống xác nhận thông tin hợp lệ.
6. Hệ thống chuyển Người dùng đến Dashboard.

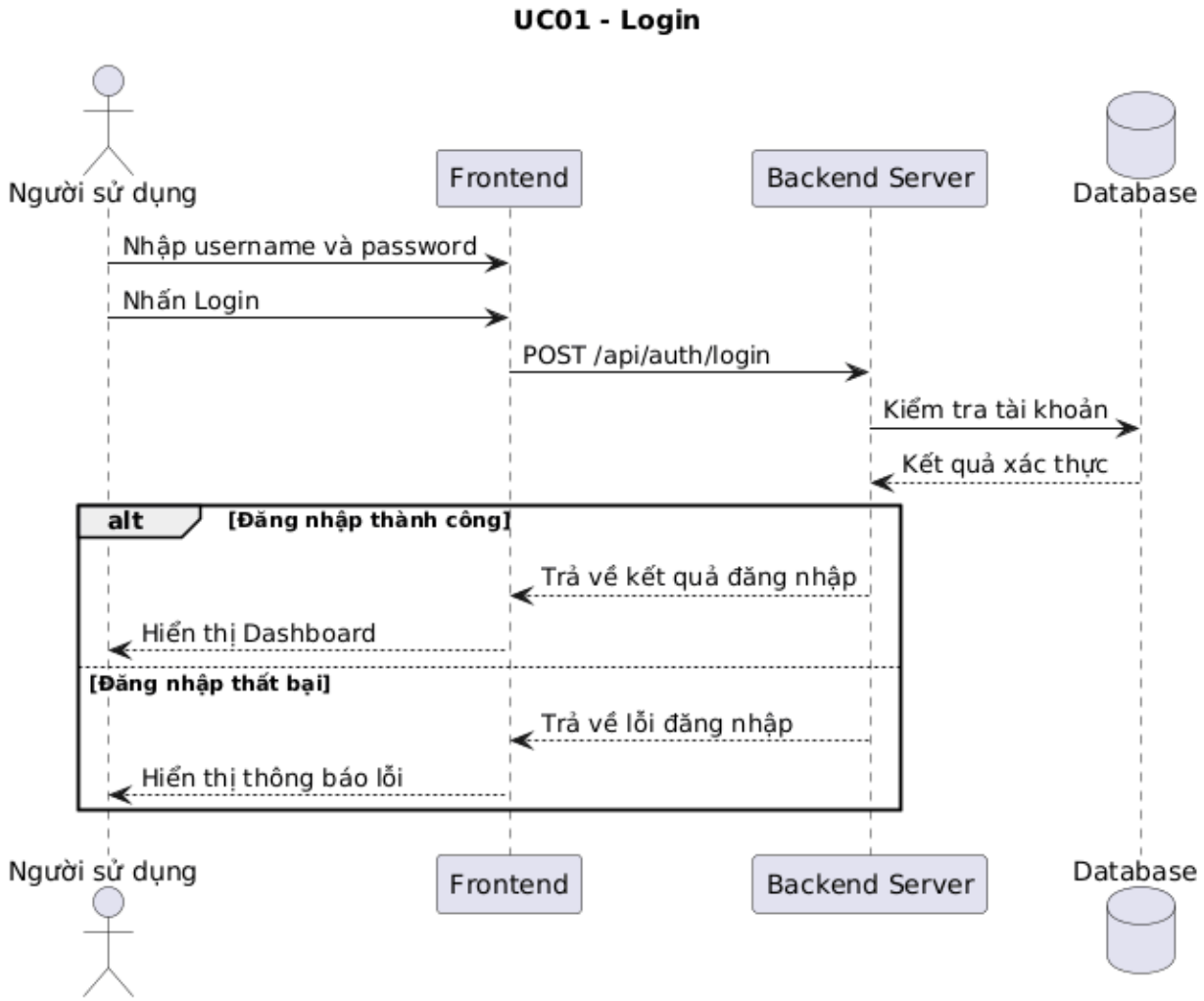
Luồng ngoại lệ:

1. Người dùng nhập username hoặc password không hợp lệ.
2. Hệ thống kiểm tra và xác định thông tin đăng nhập không chính xác.
3. Hệ thống hiển thị thông báo đăng nhập thất bại.
4. Người dùng nhập lại thông tin đăng nhập.

2.2.1.2 Biểu đồ sequence

Luồng sequence:

1. Người dùng → Frontend: Người dùng nhập username và password trên giao diện Login.
2. Người dùng → Frontend: Người dùng nhấn nút đăng nhập để gửi thông tin đăng nhập.
3. Frontend → Backend Server: Frontend gửi yêu cầu:POST /api/auth/login kèm theo thông tin đăng nhập của Người dùng.
4. Backend Server → Database: Backend nhận request, kiểm tra dữ liệu đầu vào và truy vấn bảng users để tìm tài khoản tương ứng.
5. Database → Backend Server: Database trả về thông tin tài khoản nếu tìm thấy người dùng phù hợp.
6. Backend Server: Backend tiến hành kiểm tra mật khẩu và xác định kết quả xác thực.
7. Trường hợp đăng nhập thành công: Backend trả kết quả đăng nhập thành công về Frontend.
8. Frontend → Người dùng: Frontend chuyển Người dùng sang giao diện Dashboard.
9. Trường hợp đăng nhập thất bại: Backend trả thông báo lỗi về Frontend.
10. Frontend → Người dùng: Frontend hiển thị thông báo đăng nhập không hợp lệ và cho phép Người dùng nhập lại thông tin.



Hình 3 Biểu đồ sequence usecase login

2.2.2 Usecase Dashboard & Realtime Chart

2.2.2.1 Mô tả usecase

- Cho phép Người dùng theo dõi các thông số môi trường và trạng thái thiết bị trên Dashboard.
- Lấy dữ liệu tổng quan gồm nhiệt độ, độ ẩm, ánh sáng và trạng thái LED để hiển thị trên Dashboard. Dữ liệu cảm biến mới được cập nhật từ ESP32 thông qua MQTT.

Actor: Người dùng

Tiền điều kiện: Người dùng đã đăng nhập, ESP32 kết nối MQTT Broker và hệ thống đang hoạt động.

Hậu điều kiện: Dữ liệu nhiệt độ, độ ẩm, ánh sáng được hiển thị và cập nhật trên Dashboard.

Luồng chính:

1. Người dùng truy cập Dashboard.
2. Hệ thống lấy dữ liệu cảm biến mới nhất.
3. Hệ thống hiển thị nhiệt độ, độ ẩm và ánh sáng.

4. Hệ thống hiển thị biểu đồ dữ liệu cảm biến.
5. Khi có dữ liệu mới từ ESP32, Dashboard tự động cập nhật.
6. Người dùng theo dõi sự thay đổi của dữ liệu trên giao diện.

Luồng ngoại lệ:

1. Người dùng truy cập Dashboard nhưng chưa có dữ liệu cảm biến.
2. Hệ thống hiển thị trạng thái chưa có dữ liệu.
3. Khi Backend nhận được dữ liệu mới, Dashboard được cập nhật lại.

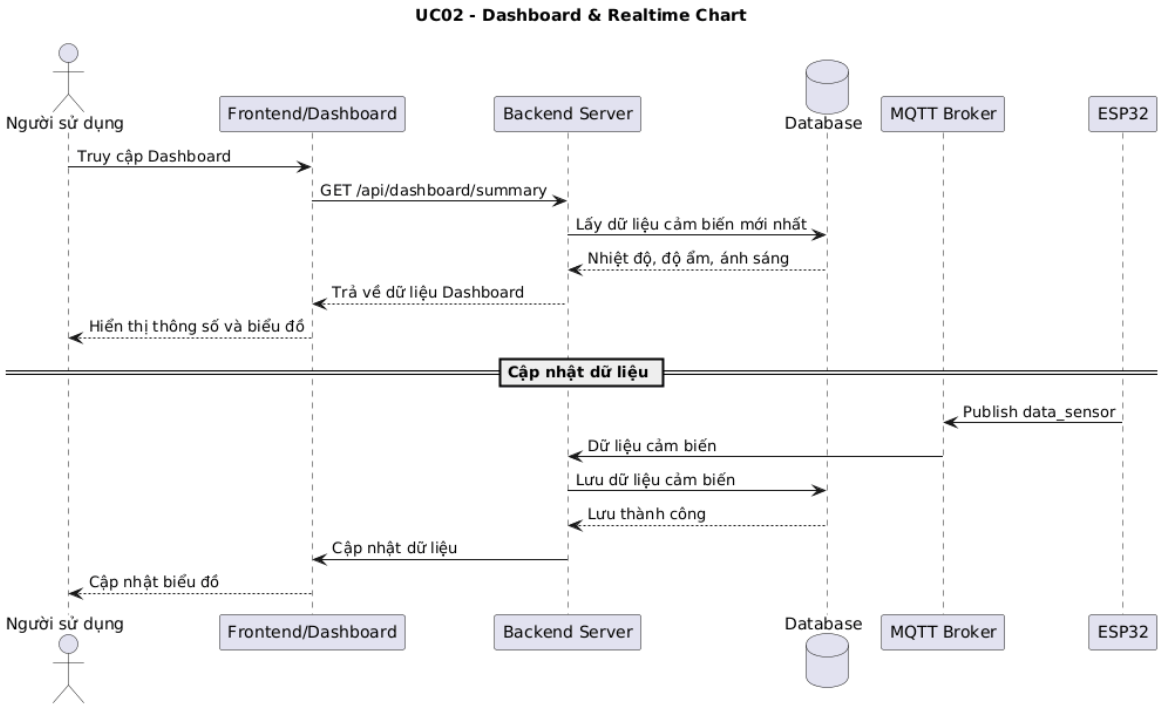
2.2.2.2 Biểu đồ sequence

Luồng sequence:

1. Người dùng → Frontend: Người dùng truy cập trang Dashboard.
2. Frontend → Backend Server: Frontend gửi yêu cầu lấy dữ liệu tổng quan:

   GET /api/dashboard/summary
3. Backend Server → Database: Backend truy vấn dữ liệu cảm biến mới nhất đã được lưu trong Database.
4. Database → Backend Server: Database trả về các giá trị nhiệt độ, độ ẩm và ánh sáng mới nhất.
5. Backend Server → Frontend: Backend trả dữ liệu Dashboard về cho Frontend.
6. Frontend → Người dùng: Frontend hiển thị các thông số cảm biến và biểu đồ dữ liệu.
7. ESP32 → MQTT Broker: Trong quá trình hệ thống hoạt động, ESP32 thu thập dữ liệu cảm biến và Publish dữ liệu lên Topic:

   data_sensor
8. MQTT Broker → Backend Server: Broker chuyển message từ Topic data_sensor đến Backend đang Subscribe Topic này.
9. Backend Server: Backend tiếp nhận dữ liệu, kiểm tra cấu trúc và xử lý các giá trị cảm biến.
10. Backend Server → Database: Backend lưu dữ liệu mới vào bảng data_sensor.
11. Database → Backend Server: Database xác nhận dữ liệu đã được lưu.
12. Backend Server → Frontend: Backend gửi dữ liệu mới đến Frontend theo cơ chế cập nhật Realtime.
13. Frontend → Người dùng: Dashboard cập nhật các giá trị hiện tại và vẽ lại biểu đồ mà không cần Người dùng tải lại trang.



Hình 4 Biểu đồ sequence usecase Usecase Dashboard & Realtime Chart

2.2.3 Usecase Sensor Data

2.2.3.1 Mô tả usecase

- Cho phép Người dùng xem, tìm kiếm và lọc dữ liệu cảm biến đã được lưu trong hệ thống.
- Lấy danh sách dữ liệu nhiệt độ, độ ẩm và ánh sáng từ Database để hiển thị trên trang Sensor Data. Người dùng có thể tìm kiếm, lọc theo loại cảm biến và phân trang dữ liệu.

Actor: Người dùng

Tiền điều kiện: Người dùng đã đăng nhập và hệ thống có dữ liệu cảm biến trong Database.

Hậu điều kiện: Người dùng xem, tìm kiếm, lọc và phân trang được dữ liệu cảm biến.

Luồng chính:

1. Người dùng truy cập trang Sensor Data.
2. Hệ thống hiển thị danh sách dữ liệu cảm biến dưới dạng bảng.
3. Người dùng có thể tìm kiếm dữ liệu.
4. Người dùng có thể lọc theo loại cảm biến như Temperature, Humidity hoặc Light.
5. Người dùng có thể chuyển trang để xem các dữ liệu tiếp theo.
6. Hệ thống hiển thị kết quả tương ứng với điều kiện được chọn.

Luồng ngoại lệ:

1. Cơ sở dữ liệu chưa có dữ liệu cảm biến hoặc không có bản ghi phù hợp với điều kiện tra cứu.

2. Hệ thống hiển thị bảng trống kèm thông báo không có dữ liệu.

3. Người dùng thay đổi hoặc xóa điều kiện tìm kiếm, lọc.

4. Hệ thống truy vấn lại và cập nhật kết quả.

2.2.3.2 Biểu đồ sequence

Luồng sequence:

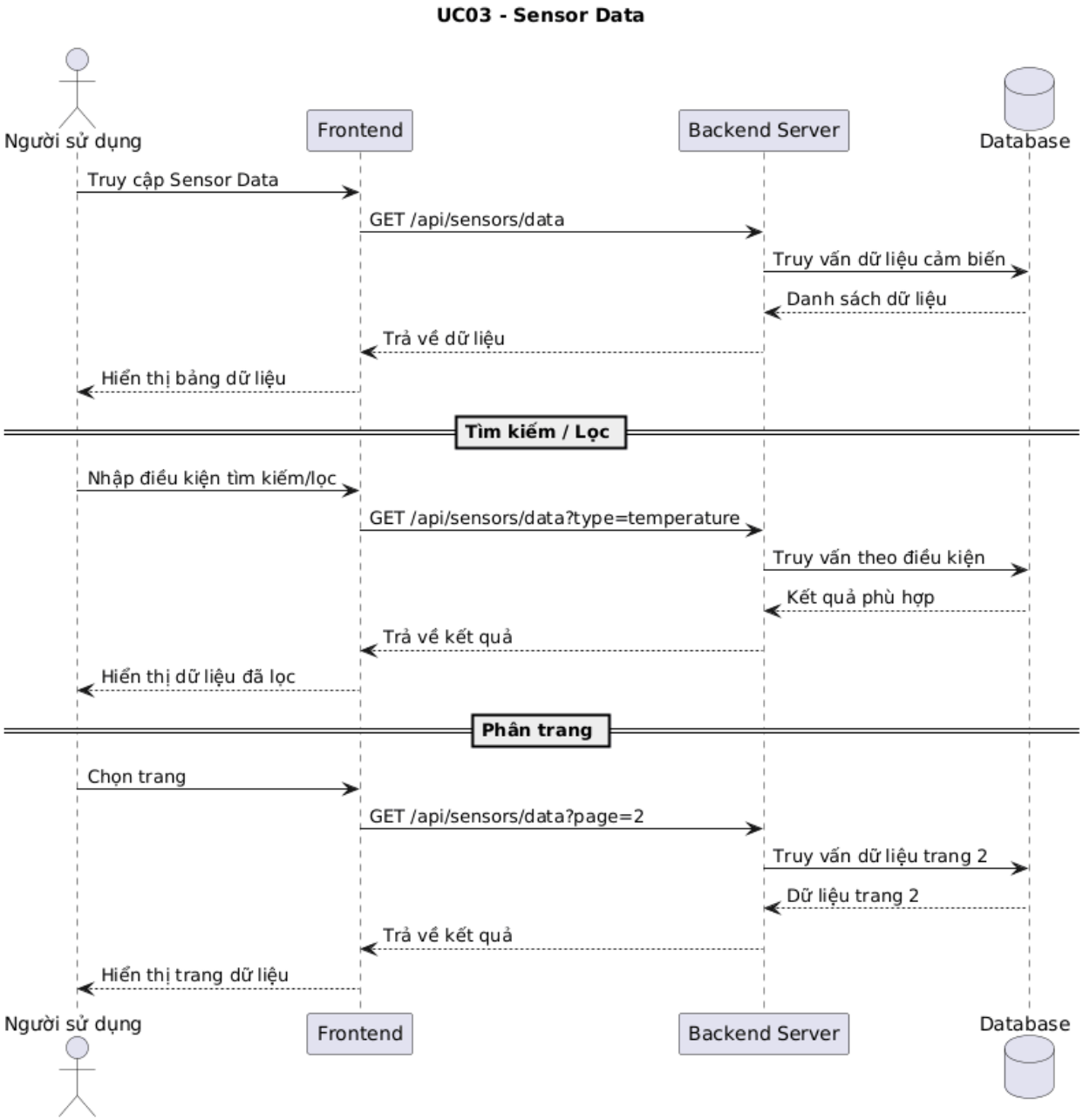
1. Người dùng → Frontend: Người dùng truy cập trang Sensor Data.
2. Frontend → Backend Server: Frontend gửi yêu cầu lấy danh sách dữ liệu cảm biến: GET /api/sensors/data
3. Backend Server → Database: Backend truy vấn bảng data_sensor và các thông tin cảm biến liên quan.
4. Database → Backend Server: Database trả về danh sách dữ liệu gồm mã cảm biến, loại cảm biến, giá trị đo, trạng thái và thời gian ghi nhận.
5. Backend Server → Frontend: Backend xử lý kết quả và trả danh sách dữ liệu về Frontend.
6. Frontend → Người dùng: Frontend hiển thị dữ liệu dưới dạng bảng.

Tìm kiếm hoặc lọc dữ liệu:

7. Người dùng → Frontend: Người dùng nhập từ khóa hoặc chọn loại cảm biến cần lọc.
8. Frontend → Backend Server: Frontend gửi yêu cầu kèm điều kiện lọc.
9. Backend Server → Database: Backend truy vấn dữ liệu theo điều kiện sensor_type = temperature.
10. Database → Backend Server: Database trả về các bản ghi phù hợp.
11. Backend Server → Frontend: Backend trả danh sách kết quả đã lọc.
12. Frontend → Người dùng: Frontend cập nhật bảng dữ liệu theo điều kiện đã chọn.

Phân trang:

13. Người dùng → Frontend: Người dùng chọn trang tiếp theo.
14. Frontend → Backend Server: Frontend gửi yêu cầu lấy dữ liệu của trang tương ứng.
15. Backend Server → Database: Backend truy vấn các bản ghi thuộc trang được yêu cầu.
16. Database → Backend Server: Database trả về dữ liệu của trang đó.
17. Backend Server → Frontend: Backend trả kết quả phân trang.
18. Frontend → Người dùng: Frontend hiển thị dữ liệu của trang được chọn.



Hình 5 Biểu đồ sequence usecase sensor data

2.2.4 Usecase Control Led

2.2.4.1 Mô tả usecase

- Cho phép Người dùng bật hoặc tắt thiết bị LED thông qua Dashboard.
- Tiếp nhận lệnh điều khiển LED từ Frontend, Backend gửi lệnh đến ESP32 thông qua MQTT. ESP32 thực hiện lệnh và phản hồi trạng thái thực tế của LED về Backend.

Actor: Người dùng

Tiền điều kiện: Người dùng đã đăng nhập, ESP32 kết nối MQTT Broker và LED được cấu hình trong hệ thống.

Hậu điều kiện: LED được điều khiển theo yêu cầu, trạng thái mới được cập nhật vào devices và lịch sử được lưu vào action_history.

Luồng chính:

1. Người dùng truy cập chức năng điều khiển LED.
2. Hệ thống lấy trạng thái hiện tại của LED.
3. Hệ thống hiển thị trạng thái LED hiện tại.
4. Người dùng chọn bật hoặc tắt LED.
5. Hệ thống ghi nhận yêu cầu điều khiển.
6. Hệ thống gửi lệnh đến thiết bị.
7. Thiết bị thực hiện lệnh và phản hồi trạng thái thực tế.
8. Hệ thống cập nhật kết quả điều khiển và trạng thái LED.
9. Hệ thống hiển thị trạng thái mới cho Người dùng.

Luồng ngoại lệ:

1. Thiết bị không thực hiện được lệnh hoặc không gửi phản hồi.
2. Hệ thống xác định thao tác điều khiển không thành công.
3. Hệ thống ghi nhận kết quả thất bại vào lịch sử điều khiển.
4. Hệ thống thông báo điều khiển thất bại cho Người dùng.

2.2.4.2 Biểu đồ sequence

Luồng sequence:

Lấy trạng thái LED hiện tại:

1. Người dùng → Frontend: Người dùng mở chức năng điều khiển LED.
2. Frontend → Backend: Frontend yêu cầu lấy trạng thái LED:

   GET /api/devices/1/status
3. Backend → Database: Backend truy vấn trạng thái LED hiện tại của thiết bị.
4. Database → Backend: Database trả về:

   led_state = OFF
5. Backend → Frontend: Backend trả trạng thái hiện tại.
6. Frontend → Người dùng: Giao diện hiển thị LED đang OFF.

Thực hiện điều khiển:

7. Người dùng → Frontend: Người dùng chọn ON.
8. Frontend → Backend: Frontend gửi yêu cầu:

   POST /api/devices/1/actions

Payload:

{

"action": "ON"

}

9. Backend → Database: Backend tạo một bản ghi mới trong action_history.

   Ban đầu:

   user_id = 1

   device_id = 1

   action = ON

   status = PENDING

   timestamp = thời điểm gửi lệnh

10. Database → Backend: Database xác nhận tạo Action History thành công.
11. Backend → Frontend: Backend trả:

    status = PENDING

12. Frontend → Người dùng: Hiển thị trạng thái đang xử lý.
13. Gửi lệnh qua MQTT
14. Backend → MQTT Broker: Backend Publish lên Topic:

    device_control

    Payload:

    {

    "device_id": 1,

    "command": "ON"

    }

15. MQTT Broker → ESP32: Broker chuyển lệnh ON đến ESP32.
16. ESP32 → LED: ESP32 điều khiển GPIO để bật LED.
17. LED → ESP32: Thao tác hoàn tất, ESP32 xác định trạng thái thực tế là ON.

Phản hồi trạng thái:

ESP32 → MQTT Broker: ESP32 Publish:

Topic: device_response

{

"device_id": 1,

"led_state": "ON"

}

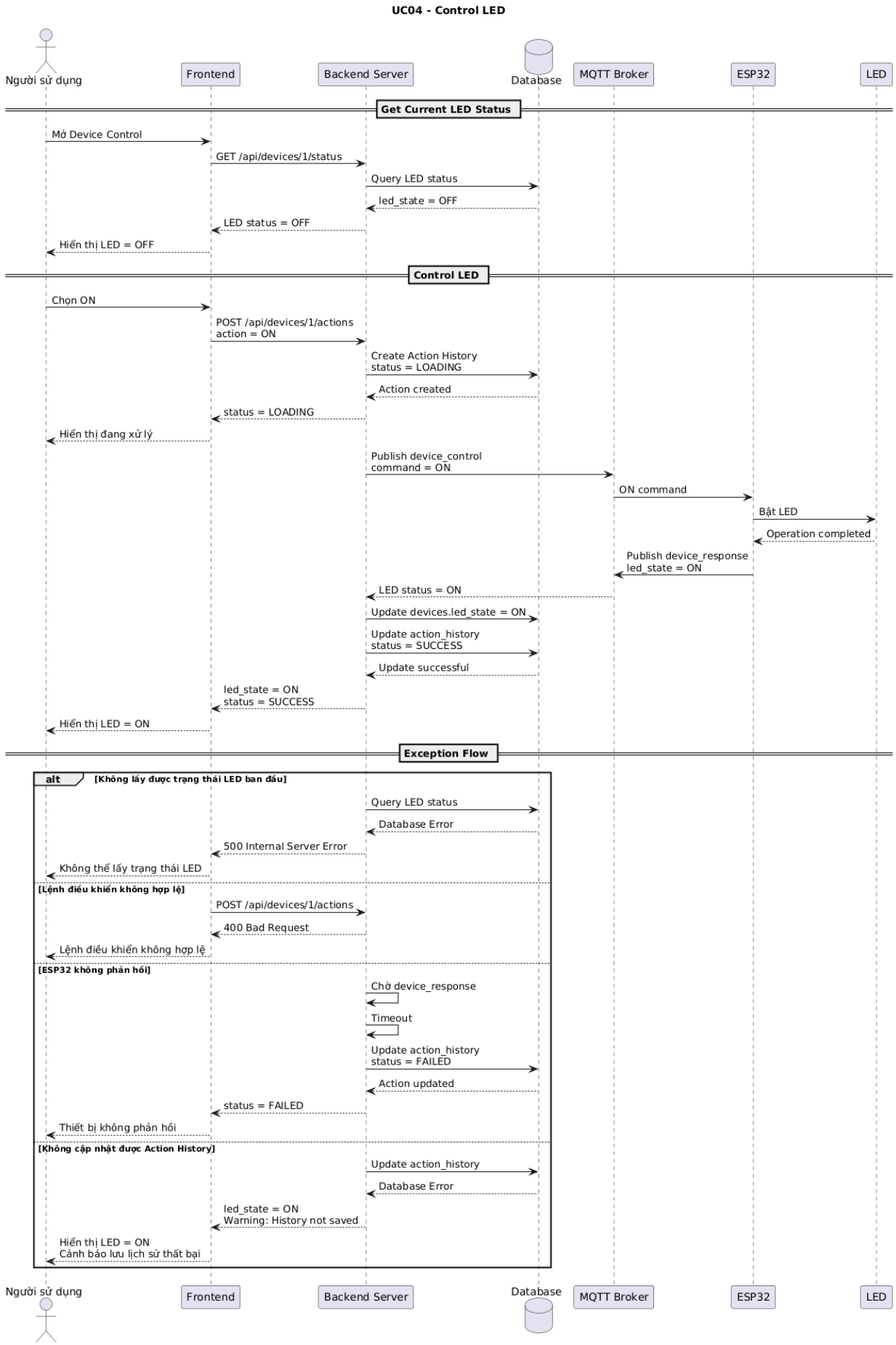
18. MQTT Broker → Backend: Backend nhận trạng thái thực tế của LED.
19. Backend → Database: Backend cập nhật devices.led_state = ON và cập nhật bản ghi action_history:

    action = ON

    status = SUCCESS
20. Database → Backend: Xác nhận cập nhật thành công.
21. Backend → Frontend: Backend gửi trạng thái mới:

    led_state = ON

    status = SUCCESS
22. Frontend: Cập nhật trạng thái LED trên Dashboard.
23. Frontend → Người dùng: Hiển thị LED đang ON.



Hình 6 Biểu đồ sequence use case control led

2.2.5 Usecase Action History

2.2.5.1 Mô tả usecase

- Cho phép người dùng xem lại lịch sử các lệnh điều khiển LED trong hệ thống.

- Hiển thị danh sách lịch sử dưới dạng bảng, hỗ trợ tìm kiếm theo thời gian, lọc theo thiết bị, thao tác, trạng thái, sắp xếp và phân trang.

Actor: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập và hệ thống đang hoạt động.

Hậu điều kiện: Hệ thống hiển thị lịch sử điều khiển phù hợp với điều kiện tra cứu hoặc thông báo không có dữ liệu.

Luồng chính:

1. Người dùng truy cập trang Action History.

2. Hệ thống lấy danh sách lịch sử điều khiển LED và thông tin phân trang.

3. Hệ thống hiển thị mã bản ghi, thiết bị, thao tác, trạng thái và thời gian ghi nhận.

4. Người dùng nhập nội dung tìm kiếm theo thời gian hoặc chọn bộ lọc thiết bị, thao tác và trạng thái.

5. Người dùng chọn thứ tự sắp xếp và số bản ghi hiển thị trên mỗi trang.

6. Khi điều kiện tra cứu thay đổi, hệ thống đưa về trang đầu và hiển thị kết quả tương ứng.

7. Người dùng chuyển trang để xem các bản ghi tiếp theo.

8. Hệ thống hiển thị trang được chọn, giữ nguyên các điều kiện tra cứu hiện tại.

Luồng ngoại lệ:

1. Cơ sở dữ liệu chưa có lịch sử điều khiển hoặc không có bản ghi phù hợp với điều kiện tra cứu.

2. Hệ thống hiển thị bảng trống kèm thông báo không có dữ liệu.

3. Người dùng thay đổi hoặc xóa điều kiện tìm kiếm, lọc.

2.2.5.2 Biểu đồ sequence

Luồng sequence:

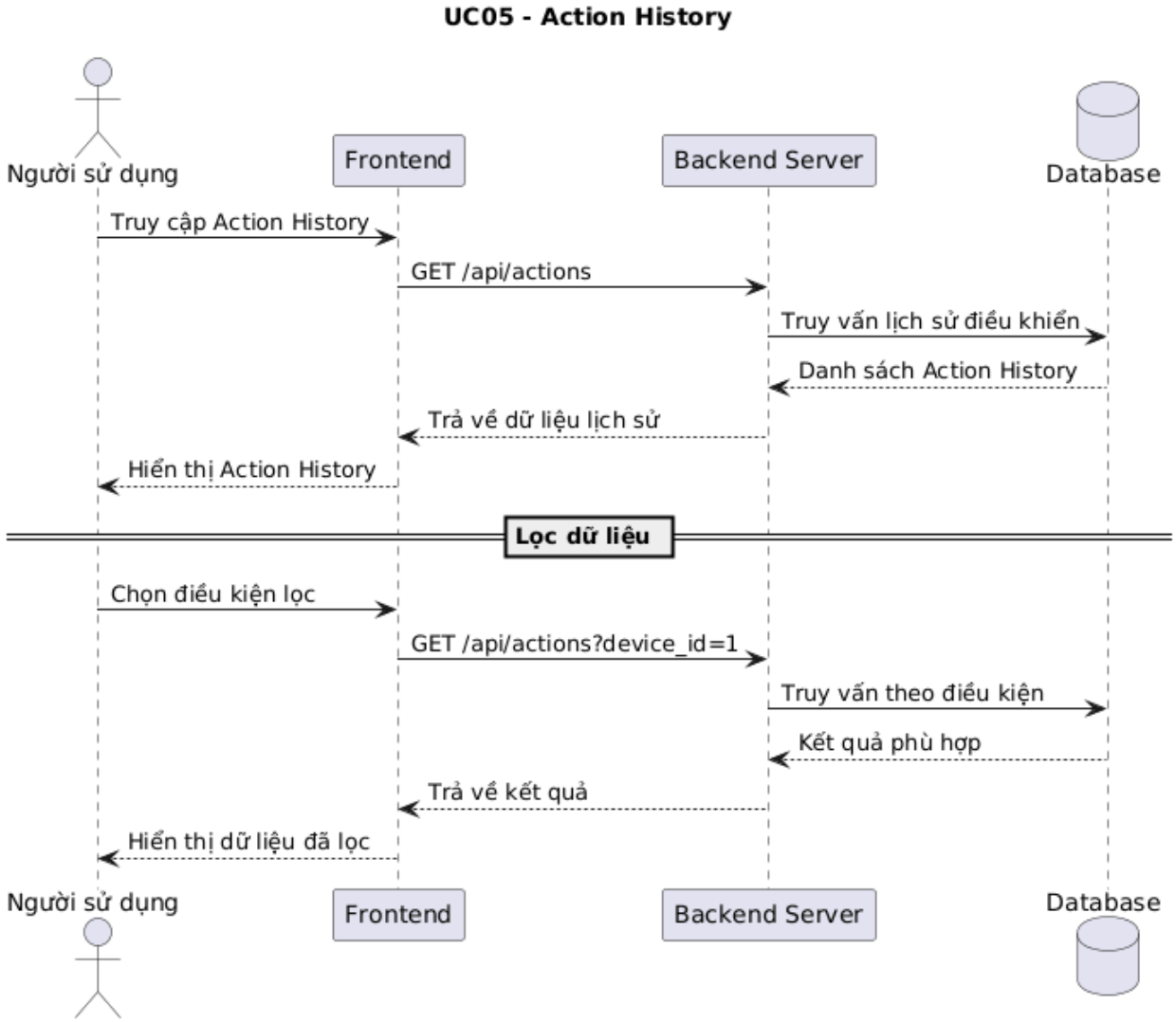
1. Người dùng → Frontend: Người dùng truy cập trang Action History.
2. Frontend → Backend Server: Frontend gửi yêu cầu lấy danh sách lịch sử:

   GET /api/actions
3. Backend Server → Database: Backend truy vấn bảng action_history.
4. Database → Backend Server: Database trả về danh sách lịch sử điều khiển.

5. Backend Server → Frontend: Backend xử lý dữ liệu và trả danh sách Action History về Frontend.
6. Frontend → Người dùng: Frontend hiển thị dữ liệu dưới dạng bảng.

Lọc dữ liệu:

7. Người dùng → Frontend: Người dùng chọn điều kiện lọc, ví dụ theo thiết bị.
8. Frontend → Backend Server: Frontend gửi yêu cầu: GET /api/actions?device_id=1
9. Backend Server → Database: Backend truy vấn action_history theo device_id.
10. Database → Backend Server: Database trả về các bản ghi phù hợp.
11. Backend Server → Frontend: Backend trả dữ liệu đã lọc.
12. Frontend → Người dùng: Frontend cập nhật bảng Action History.



Hình 7 Biểu đồ sequence usecase Action History

2.2.6 Usecase Profile

2.2.6.1 Mô tả usecase

- Cho phép người dùng xem thông tin cá nhân và các liên kết tài liệu liên quan gồm GitHub, Figma, tài liệu API và báo cáo PDF.

- Người dùng có thể chọn các liên kết để truy cập tài liệu tương ứng.

Actor: Người dùng.

Tiền điều kiện: Người dùng đã đăng nhập và thông tin tài khoản tồn tại trong cơ sở dữ liệu.

Hậu điều kiện: Thông tin Profile được hiển thị trên giao diện.

Luồng chính:

1. Người dùng truy cập trang Profile.
2. Hệ thống lấy thông tin tài khoản theo ID của người dùng đang đăng nhập.
3. Hệ thống hiển thị thông tin cá nhân và các liên kết tài liệu.
4. Người dùng xem thông tin trên giao diện.
5. Người dùng chọn liên kết GitHub, Figma, tài liệu API hoặc báo cáo PDF nếu cần.
6. Trình duyệt mở tài liệu tương ứng trong tab mới.

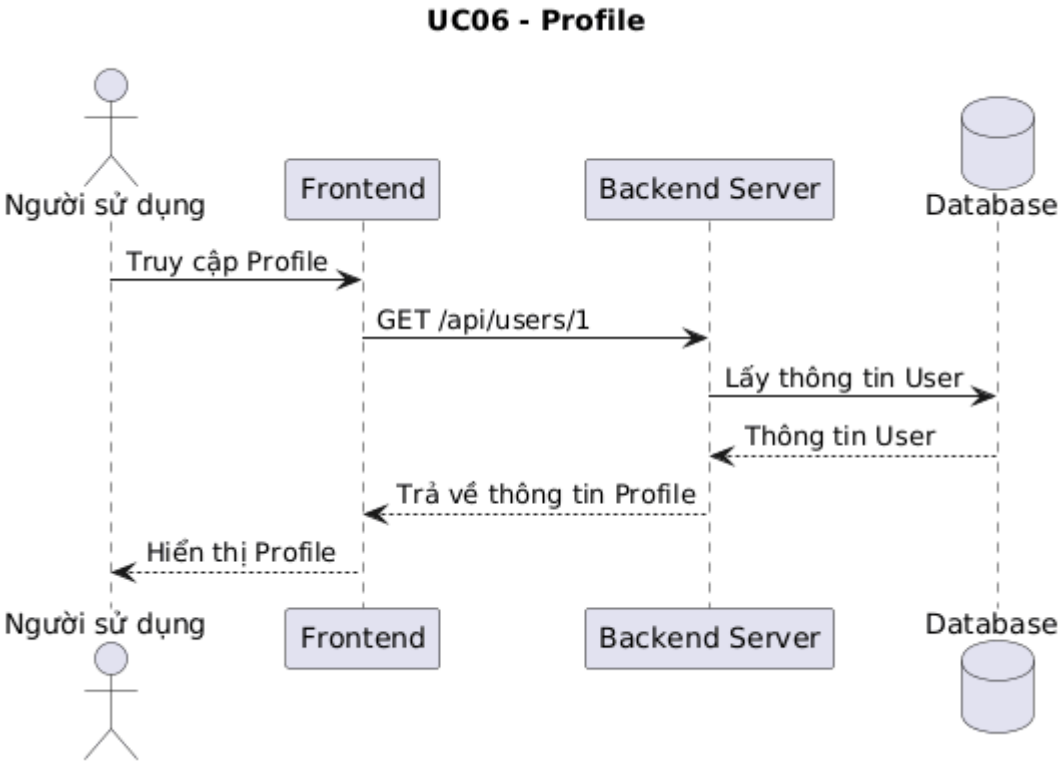
Luồng ngoại lệ:

1. Hệ thống không tìm thấy tài khoản hoặc không tải được thông tin Profile.
2. Hệ thống hiển thị thông báo lỗi.
3. Người dùng truy cập lại trang Profile để tải lại thông tin.

2.2.6.2 Biểu đồ sequence

Luồng sequence:

1. Người dùng → Frontend: Người dùng truy cập trang Profile.
2. Frontend → Backend Server: Frontend gửi yêu cầu lấy thông tin Người dùng: GET /api/users/1
3. Backend Server → Database: Backend truy vấn bảng users theo id = 1.
4. Database → Backend Server: Database trả về thông tin người dùng.
5. Backend Server → Frontend: Backend trả thông tin Profile về Frontend.
6. Frontend → Người dùng: Frontend hiển thị thông tin Profile trên giao diện.



Hình 8 Biểu đồ sequence usecase profile

CHƯƠNG 3. XÂY DỰNG HỆ THỐNG

3.1 Xây dựng phần cứng

3.1.1 Các thành phần phần cứng

Phần cứng của hệ thống gồm ESP32, cảm biến DHT11, cảm biến ánh sáng LDR và ba đèn LED. Các thành phần này thực hiện hai nhiệm vụ chính là thu thập dữ liệu môi trường và tiếp nhận yêu cầu điều khiển từ người dùng.

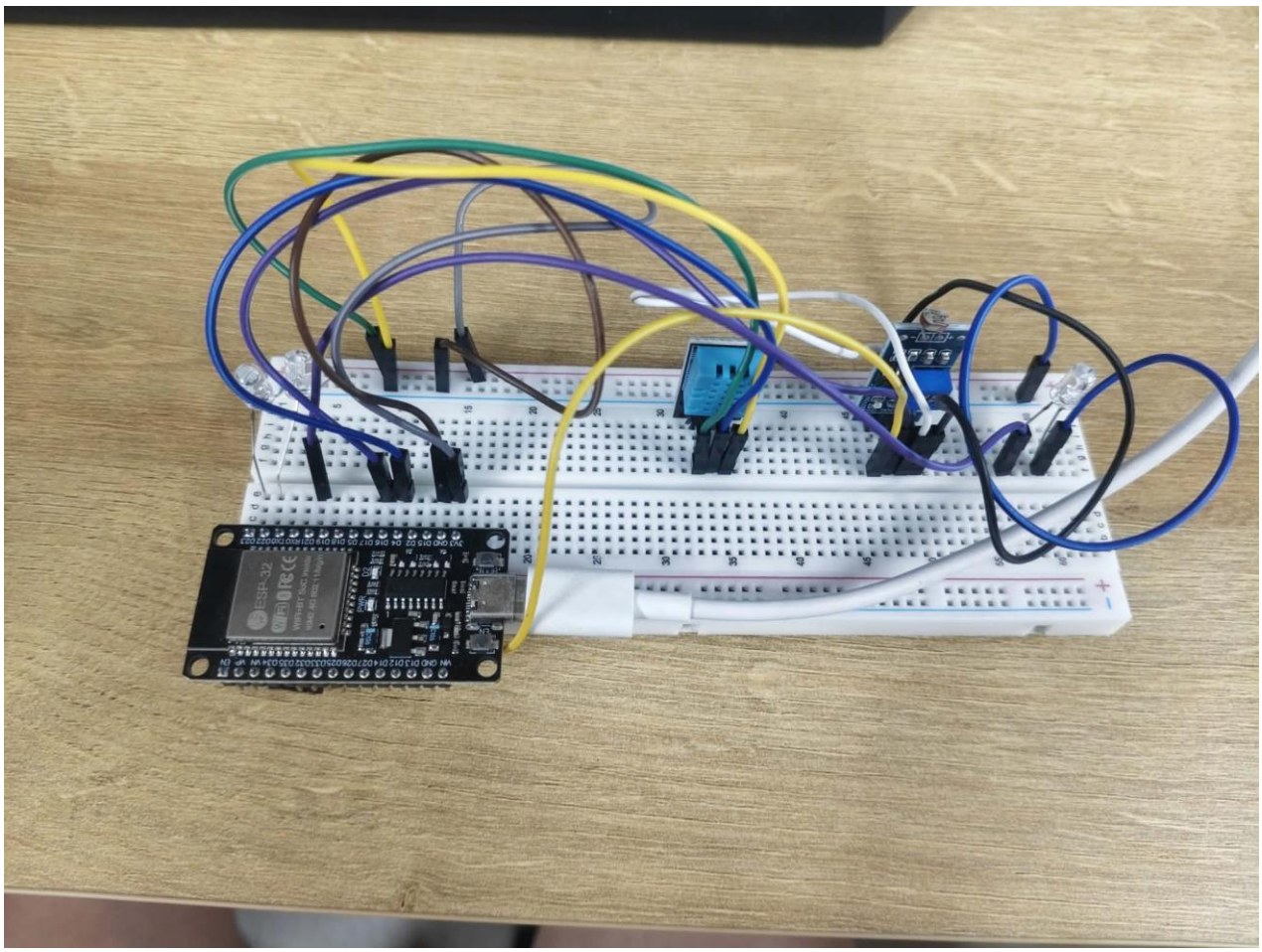
ESP32 được sử dụng làm bộ điều khiển trung tâm. Khả năng kết nối Wi-Fi cho phép thiết bị trao đổi dữ liệu với máy chủ mà không cần bổ sung module mạng riêng. Các chân GPIO đáp ứng nhu cầu kết nối cảm biến và điều khiển độc lập ba LED, phù hợp với quy mô mô hình của đề tài.

DHT11 được lựa chọn để thu thập nhiệt độ và độ ẩm. Việc cung cấp hai thông số trên cùng một cảm biến giúp đơn giản hóa kết nối phần cứng. Cảm biến này phục vụ mục đích theo dõi môi trường ở mức cơ bản trong mô hình thử nghiệm.

LDR được sử dụng để ghi nhận sự thay đổi của ánh sáng. Tín hiệu từ LDR được ESP32 đọc thông qua chân analog. Trong chương trình hiện tại, dữ liệu ánh sáng là giá trị ADC, chưa được hiệu chuẩn và chuyển đổi thành độ rọi thực tế. Do đó, giá trị này được dùng để theo dõi mức ánh sáng tương đối.

Ba LED đóng vai trò thiết bị chấp hành, mô phỏng các thiết bị cần bật hoặc tắt từ xa. Mỗi LED được điều khiển bằng một chân GPIO riêng và được quản lý bằng một ID tương ứng.

Thành phần	**Chân kết nối trên ESP32**
DHT11	GPIO4
LDR	GPIO34
LED 1	GPIO21
LED 2	GPIO22
LED 3	GPIO23



Hình 9 Thiết kế mạch

3.1.2 Ngôn ngữ và thư viện sử dụng

Chương trình cho ESP32 được xây dựng bằng C/C++ trong môi trường Arduino IDE. Cách triển khai này phù hợp với việc điều khiển các chân GPIO, đọc cảm biến và xử lý kết nối mạng trên vi điều khiển.

Thư viện WiFi hỗ trợ kết nối ESP32 với mạng không dây. Thư viện DHT cung cấp các hàm đọc nhiệt độ và độ ẩm. PubSubClient được sử dụng để thiết lập kết nối MQTT, gửi dữ liệu cảm biến và nhận lệnh điều khiển từ máy chủ.

Việc sử dụng các thư viện giúp giảm phần xử lý giao tiếp ở mức thấp, đồng thời tập trung chương trình vào các nhiệm vụ của hệ thống như kiểm tra dữ liệu, gửi bản tin, điều khiển LED và phản hồi kết quả.

3.1.3 Hoạt động của phần cứng

Sau khi khởi động, ESP32 thiết lập các chân kết nối, khởi tạo cảm biến và kết nối Wi-Fi. Tiếp đó, thiết bị kết nối với MQTT Broker, đăng ký nhận lệnh điều khiển và yêu cầu đồng bộ trạng thái LED.

Dữ liệu nhiệt độ, độ ẩm và ánh sáng được đọc theo chu kỳ 5 giây. Nếu dữ liệu từ DHT11 không hợp lệ, chương trình bỏ qua lần gửi đó. Các giá trị hợp lệ được đóng gói thành JSON và gửi đến topic data_sensor.

Khi nhận lệnh trên topic device_control, ESP32 xác định LED cần điều khiển và thực hiện thao tác ON hoặc OFF. Sau khi xử lý, thiết bị gửi phản hồi qua topic device_status, kèm mã lệnh để Backend đối chiếu với yêu cầu đang chờ.

Trong quá trình hoạt động, ESP32 kiểm tra kết nối Wi-Fi và MQTT để thực hiện kết nối lại khi cần. Khi kết nối MQTT được khôi phục, thiết bị đồng bộ trạng thái ba LED trước khi tiếp tục xử lý các lệnh thông thường.

3.2 Frontend

3.2.1 HTML, CSS và JavaScript

Frontend của hệ thống được xây dựng bằng HTML, CSS và JavaScript, hoạt động trên trình duyệt web. Ba công nghệ này đảm nhiệm các vai trò khác nhau trong việc tổ chức giao diện và xử lý tương tác với người dùng.

HTML được sử dụng để xây dựng cấu trúc trang, bao gồm biểu mẫu đăng nhập, khu vực hiển thị thông số, bảng dữ liệu và các nút điều khiển. CSS định dạng bố cục, màu sắc và kiểu hiển thị, giúp các thành phần được trình bày thống nhất giữa các trang.

JavaScript xử lý các thao tác trên giao diện, gửi yêu cầu đến Backend và cập nhật nội dung khi nhận được kết quả. Nhờ đó, người dùng có thể tra cứu dữ liệu hoặc điều khiển LED mà không cần tải lại toàn bộ trang sau mỗi thao tác.

Đề tài lựa chọn JavaScript thuần vì quy mô giao diện gồm số lượng chức năng tương đối rõ ràng, chưa cần đến cơ chế quản lý phức tạp của một framework. Cách triển khai này đáp ứng yêu cầu của mô hình, đồng thời giúp thuận tiện trong việc theo dõi và kiểm tra quá trình trao đổi dữ liệu giữa Frontend với Backend.

3.2.2 Fetch API

Fetch API được sử dụng để gửi yêu cầu HTTP từ trình duyệt đến Backend. Dữ liệu trao đổi được biểu diễn dưới dạng JSON, phục vụ các chức năng đăng nhập, lấy thông tin Dashboard, tra cứu lịch sử và gửi lệnh điều khiển.

Fetch API được lựa chọn vì có sẵn trong trình duyệt, không yêu cầu bổ sung thư viện cho các yêu cầu HTTP thông thường. Việc kết hợp với cú pháp async/await giúp tổ chức các bước gửi yêu cầu, nhận phản hồi và xử lý lỗi theo trình tự dễ theo dõi.

Đối với những API cần xác thực, Frontend gửi JWT trong header Authorization. Cách trao đổi này cho phép Backend kiểm tra quyền truy cập trước khi xử lý yêu cầu.

3.2.3 Chart.js

Chart.js được sử dụng để biểu diễn dữ liệu cảm biến dưới dạng biểu đồ đường. Các giá trị nhiệt độ, độ ẩm và ánh sáng được thể hiện theo thời gian, giúp người dùng quan sát sự thay đổi qua nhiều lần đo.

Thư viện được lựa chọn vì đáp ứng nhu cầu xây dựng biểu đồ trong ứng dụng web mà không phải tự xử lý toàn bộ quá trình vẽ. Chart.js hỗ trợ cấu hình trục, chú thích và cập nhật dữ liệu trên biểu đồ hiện có.

Trong hệ thống, biểu đồ sử dụng tối đa 20 lần đo gần nhất. Việc giới hạn số điểm giúp tập trung vào diễn biến gần thời điểm quan sát và giữ giao diện dễ đọc.

3.2.4 Socket.IO Client

Socket.IO Client được sử dụng để tiếp nhận các sự kiện từ Backend trong quá trình hệ thống hoạt động. Những sự kiện này thông báo khi có dữ liệu cảm biến mới, thay đổi trạng thái kết nối hoặc kết quả điều khiển LED.

Công nghệ này được lựa chọn vì dữ liệu có thể phát sinh từ phần cứng mà không bắt đầu bằng thao tác trên trình duyệt. Nếu chỉ sử dụng các yêu cầu HTTP do người dùng thực hiện, giao diện sẽ không nhận biết ngay những thay đổi đó.

Socket.IO được sử dụng kết hợp với Fetch API. Fetch API phục vụ truy vấn và gửi yêu cầu, còn Socket.IO tiếp nhận thông báo cập nhật. Khi nhận sự kiện cảm biến mới, Frontend gọi lại API để lấy dữ liệu tổng quan và làm mới giao diện.

3.3 Backend

3.3.1 JavaScript và Node.js

Backend được xây dựng bằng JavaScript trên nền tảng Node.js. Thành phần này tiếp nhận yêu cầu từ Frontend, xử lý dữ liệu từ ESP32, truy cập cơ sở dữ liệu và trả kết quả cho người dùng.

Node.js được lựa chọn vì hệ thống có nhiều hoạt động trao đổi dữ liệu qua mạng, gồm HTTP, MQTT và Socket.IO. Cơ chế xử lý bất đồng bộ cho phép tổ chức các tác vụ này mà không phải chờ một thao tác vào/ra hoàn tất trước khi tiếp nhận mọi công việc khác.

Việc sử dụng JavaScript cho cả Frontend và Backend giúp thống nhất ngôn ngữ phát triển. Dữ liệu JSON từ trình duyệt và thiết bị có thể được chuyển thành các đối tượng để kiểm tra, xử lý và truyền giữa các thành phần.

3.3.2 Express

Express được sử dụng để xây dựng các API HTTP. Thư viện cung cấp cơ chế định tuyến yêu cầu theo đường dẫn, phương thức và các bước xử lý trung gian.

Express được lựa chọn vì phù hợp với Backend cung cấp API cho một giao diện web độc lập. Các chức năng như xác thực, kiểm tra yêu cầu và xử lý lỗi có thể được áp dụng theo một quy trình chung, hạn chế lặp lại giữa các API.

Việc sử dụng Express cũng tạo điều kiện phân chia trách nhiệm trong Backend. Phần tiếp nhận yêu cầu được tách khỏi xử lý nghiệp vụ và truy cập cơ sở dữ liệu, giúp thuận tiện khi bảo trì hoặc bổ sung chức năng.

3.3.3 MQTT.js và Mosquitto

Backend sử dụng MQTT.js để kết nối với MQTT Broker Mosquitto. Thông qua kết nối này, Backend nhận dữ liệu cảm biến, nhận phản hồi từ ESP32 và gửi lệnh điều khiển LED.

MQTT được lựa chọn vì phù hợp với mô hình trao đổi bản tin giữa thiết bị và máy chủ. Các thành phần gửi hoặc đăng ký nhận dữ liệu theo topic, cho phép phân biệt dữ liệu cảm biến, lệnh điều khiển và thông báo trạng thái.

MQTT.js cung cấp khả năng kết nối MQTT trong môi trường Node.js, giúp Backend xử lý bản tin trong cùng nền tảng với các API HTTP. Mosquitto đảm nhiệm vai trò trung gian tiếp nhận và chuyển bản tin giữa ESP32 với Backend.

3.3.4 Socket.IO

Socket.IO được sử dụng ở Backend để gửi sự kiện đến các trình duyệt đã kết nối. Sau khi xử lý dữ liệu hoặc phản hồi từ thiết bị, Backend có thể thông báo cho Frontend cập nhật giao diện.

Công nghệ này được lựa chọn nhằm đáp ứng yêu cầu theo dõi dữ liệu và kết quả điều khiển trong quá trình vận hành. Người dùng không phải chủ động gửi yêu cầu kiểm tra sau mỗi thay đổi của thiết bị.

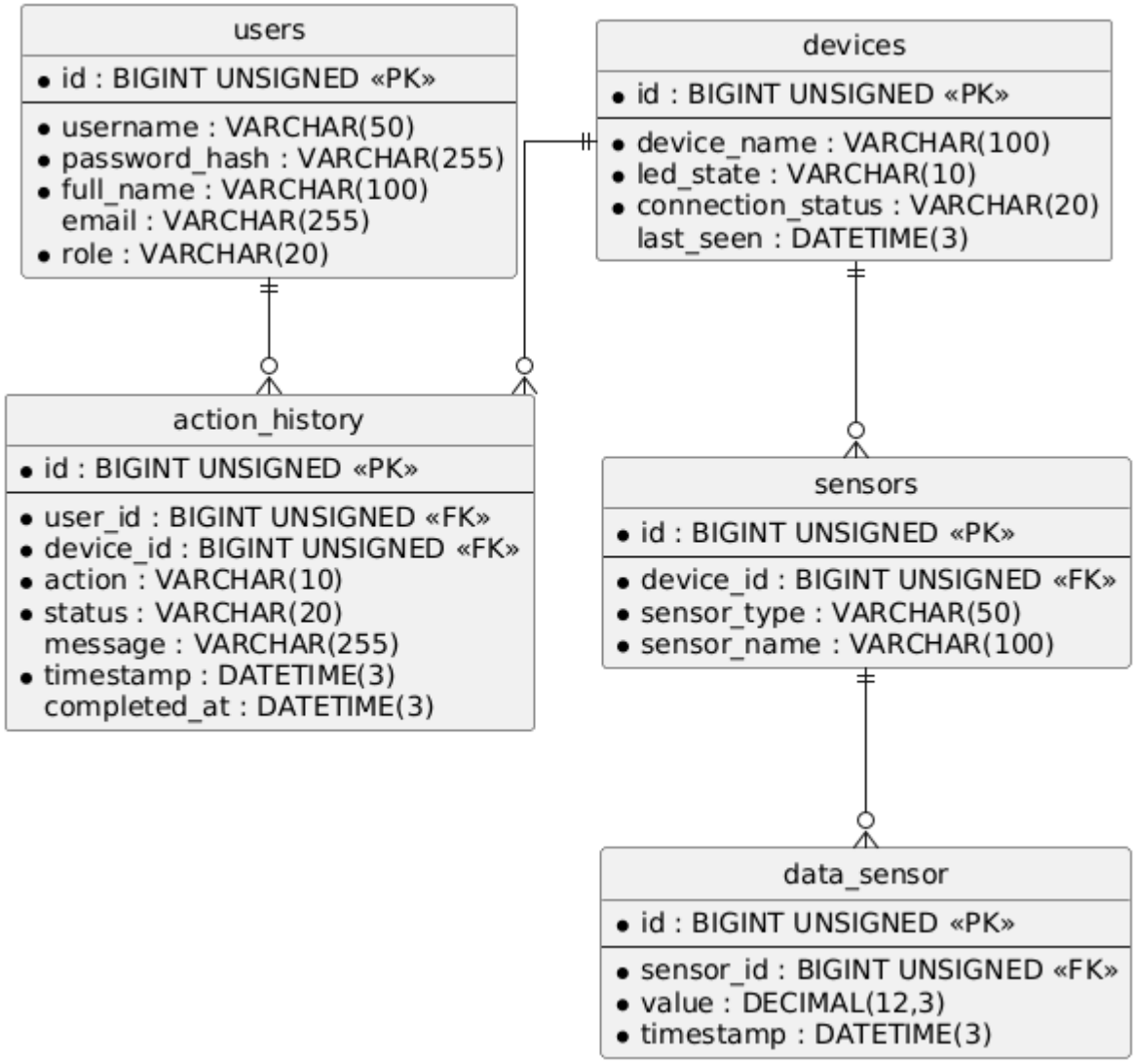
Socket.IO đảm nhiệm kết nối giữa Backend với Frontend, còn MQTT phục vụ trao đổi giữa Backend với ESP32. Hai công nghệ được sử dụng ở các đoạn truyền thông khác nhau, phù hợp với vai trò của từng thành phần.

3.4 Database

Hệ thống sử dụng MySQL để lưu trữ thông tin tài khoản, thiết bị, dữ liệu cảm biến và lịch sử điều khiển. MySQL được lựa chọn vì dữ liệu có cấu trúc và có quan hệ rõ ràng, đồng thời đáp ứng các yêu cầu tìm kiếm, lọc, sắp xếp và phân trang.

Cơ sở dữ liệu được đặt tên là iot_system, gồm năm bảng

Bảng	Mô tả
users	Lưu thông tin tài khoản, gồm tên đăng nhập, mật khẩu đã băm, họ tên, email và vai trò.
devices	Lưu thông tin thiết bị, trạng thái bật/tắt LED, trạng thái kết nối và thời điểm hoạt động gần nhất.
sensors	Lưu danh mục cảm biến, gồm mã cảm biến, thiết bị liên quan, loại cảm biến và tên cảm biến.
data_sensor	Lưu dữ liệu đo, gồm mã bản ghi, mã cảm biến, giá trị đo và thời gian ghi nhận.
action_history	Lưu lịch sử điều khiển LED, gồm mã lệnh, người thực hiện, thiết bị được điều khiển, thao tác ON/OFF, trạng thái xử lý, thông báo kết quả, thời gian tạo lệnh và thời gian hoàn tất.



Hình 10 Các bảng CSDL

3.4.1 Bảng users

Thuộc tính	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa**
id	BIGINT UNSIGNED	Khóa chính, tự tăng	Mã người dùng
username	VARCHAR(50)	NOT NULL, UNIQUE	Tên đăng nhập
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã băm
full_name	VARCHAR(100)	NOT NULL	Họ tên
email	VARCHAR(255)	Cho phép NULL, UNIQUE	Địa chỉ email
role	VARCHAR(20)	NOT NULL, mặc định 'USER'	Vai trò tài khoản

3.4.2 Bảng devices

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	BIGINT UNSIGNED	Khóa chính, tự tăng	Mã thiết bị
device_name	VARCHAR(100)	NOT NULL	Tên thiết bị
led_state	VARCHAR(10)	NOT NULL	Trạng thái bật/tắt LED
connection_status	VARCHAR(20)	NOT NULL	Trạng thái kết nối
last_seen	DATETIME(3)	Cho phép NULL	Thời điểm hoạt động gần nhất

3.4.3 Bảng sensors

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	BIGINT UNSIGNED	Khóa chính, tự tăng	Mã cảm biến
device_id	BIGINT UNSIGNED	NOT NULL, khóa ngoại → devices(id)	Thiết bị gắn với cảm biến
sensor_type	VARCHAR(50)	NOT NULL	Loại cảm biến
sensor_name	VARCHAR(100)	NOT NULL	Tên cảm biến

3.4.4 Bảng data_sensor

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	BIGINT UNSIGNED	Khóa chính, tự tăng	Mã bản ghi đo
sensor_id	BIGINT UNSIGNED	NOT NULL, khóa ngoại → sensors(id)	Cảm biến cung cấp dữ liệu
value	DECIMAL(12,3)	NOT NULL	Giá trị đo
timestamp	DATETIME(3)	NOT NULL	Thời điểm ghi nhận

3.4.5 Bảng action_history

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	BIGINT UNSIGNED	Khóa chính, tự tăng	Mã lệnh điều khiển
user_id	BIGINT UNSIGNED	NOT NULL, FK(users(id))	Người gửi lệnh
device_id	BIGINT UNSIGNED	NOT NULL, FK(devices(id))	Thiết bị được điều khiển
action	VARCHAR(10)	NOT NULL	Thao tác bật/tắt
status	VARCHAR(20)	NOT NULL	Trạng thái xử lý lệnh

message	VARCHAR(255)	Cho phép NULL	Thông báo kết quả hoặc nguyên nhân lỗi
timestamp	DATETIME(3)	NOT NULL	Thời điểm tạo lệnh
completed_at	DATETIME(3)	Cho phép NULL	Thời điểm hoàn tất xử lý

3.5 API

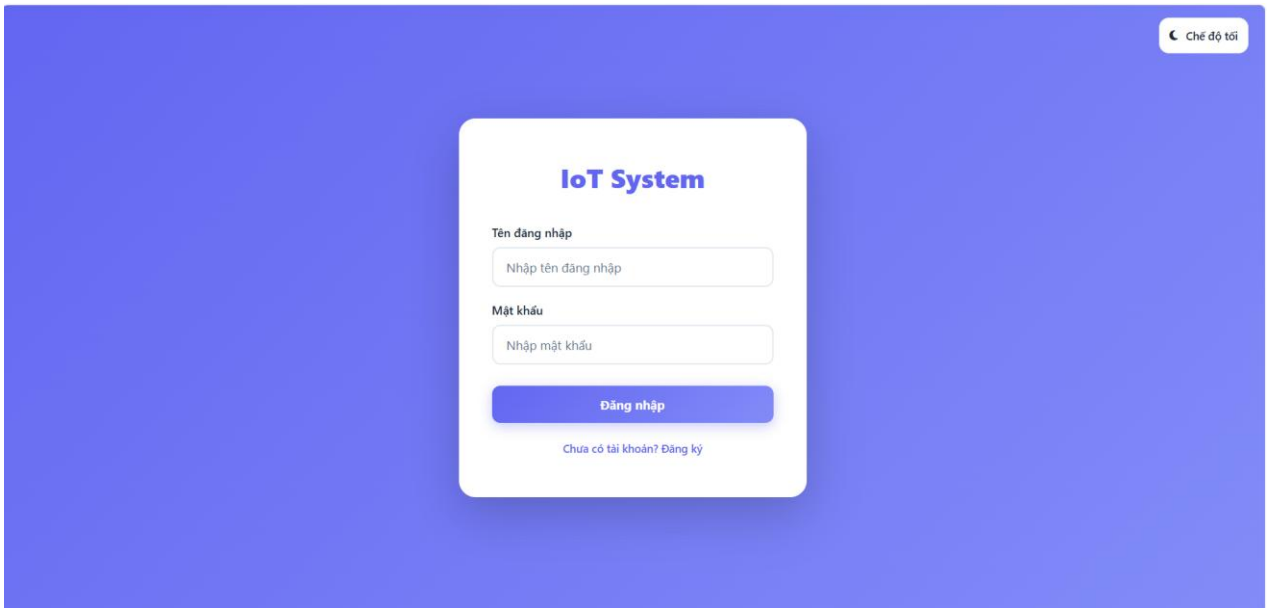
Hệ thống cung cấp các API phục vụ đăng nhập, giám sát dữ liệu, điều khiển LED và tra cứu lịch sử. Danh sách API chính được trình bày trong bảng dưới đây.

Phương thức	**Đường dẫn**	**Chức năng**
POST	/api/auth/login	Đăng nhập, trả token và thông tin người dùng
GET	/api/users/:userId	Lấy thông tin Profile và các liên kết tài liệu
GET	/api/dashboard/summary	Lấy dữ liệu tổng quan Dashboard
GET	/api/sensors/data	Tra cứu dữ liệu cảm biến
GET	/api/devices/:deviceId/status	Lấy trạng thái LED và kết nối đã lưu
POST	/api/devices/:deviceId/actions	Gửi lệnh bật hoặc tắt LED
GET	/api/actions	Tra cứu lịch sử điều khiển LED

CHƯƠNG 4. KẾT QUẢ THỰC HIỆN

4.1 Giao diện login

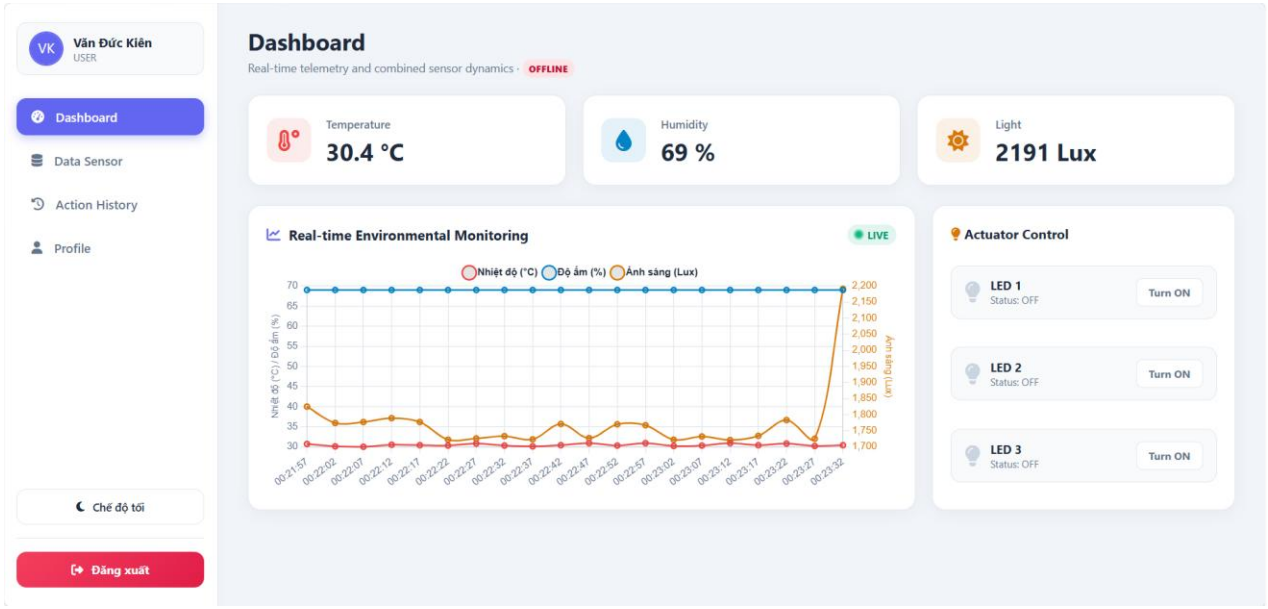
Trang Login cho phép người dùng nhập tên đăng nhập và mật khẩu để truy cập hệ thống. Khi thông tin hợp lệ, giao diện hiển thị Dashboard, nếu thông tin không chính xác, hệ thống hiển thị thông báo lỗi và cho phép người dùng nhập lại



Hình 11 Giao diện login

4.2 Giao diện dashboard

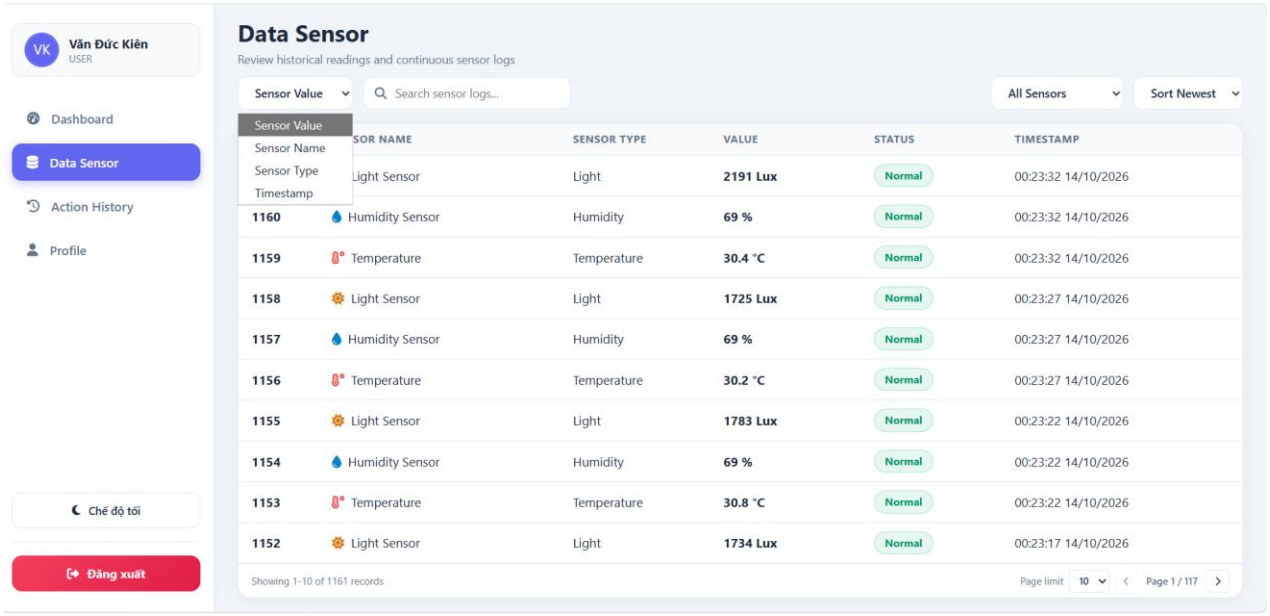
Trang Dashboard hiển thị nhiệt độ, độ ẩm, ánh sáng, trạng thái kết nối thiết bị và trạng thái ba LED. Biểu đồ thể hiện tối đa 20 lần đo gần nhất và được cập nhật khi có dữ liệu mới. Người dùng có thể theo dõi các thông số và thực hiện thao tác điều khiển ngay trên giao diện.



Hình 12 Giao diện dashboard

4.3 Giao diện Data Sensor

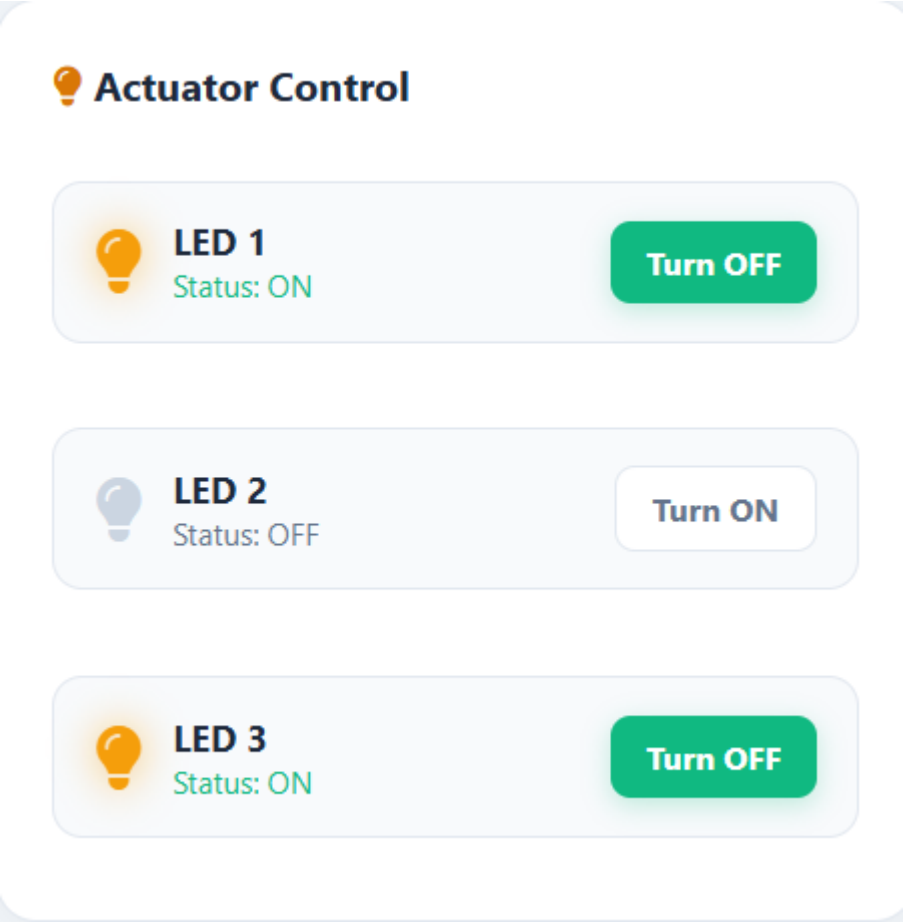
Trang Sensor Data hiển thị các bản ghi cảm biến đã lưu. Người dùng có thể lựa chọn thuộc tính tìm kiếm gồm giá trị đo, tên cảm biến, loại cảm biến hoặc thời gian, đồng thời lọc riêng dữ liệu Temperature, Humidity hoặc Light. Kết quả được sắp xếp theo thứ tự mới nhất hoặc cũ nhất và hiển thị theo từng trang.



Hình 13 Giao diện Data Sensor

4.4 Giao diện Control Led

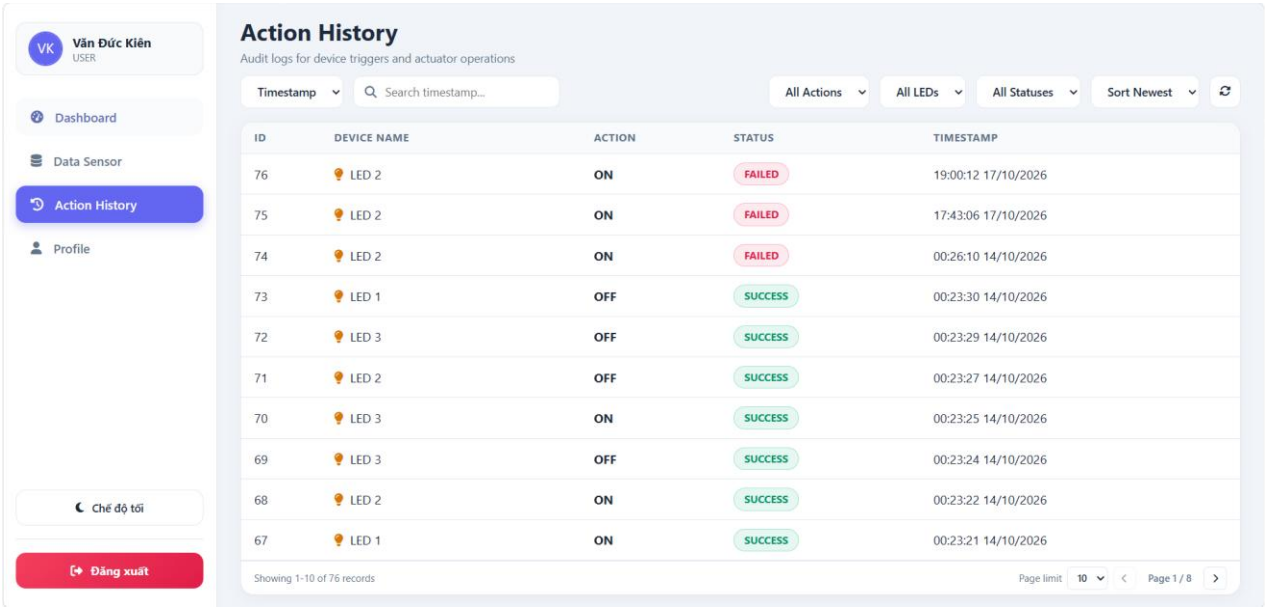
Khu vực Control LED cho phép người dùng bật hoặc tắt độc lập từng LED trên Dashboard. Khi gửi lệnh, nút điều khiển của LED tương ứng chuyển sang trạng thái chờ xử lý. Giao diện cập nhật trạng thái đèn khi nhận kết quả thành công, nếu lệnh thất bại, trạng thái đã ghi nhận trước đó được giữ nguyên



Hình 14 Giao diện Control led

4.5 Giao diện Action History

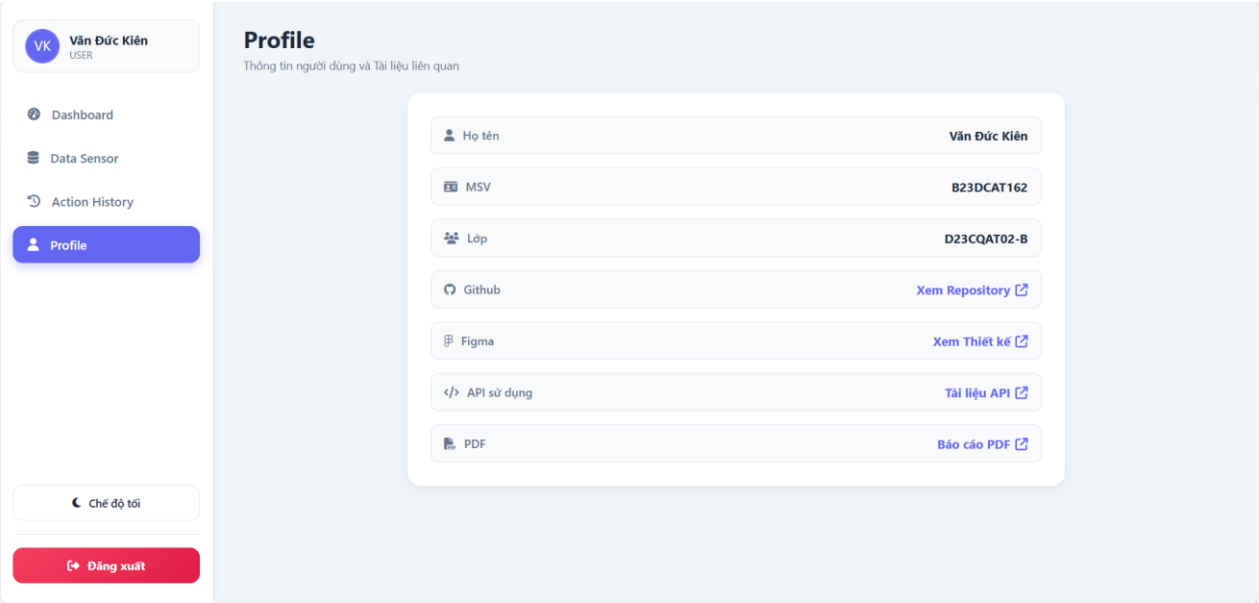
Trang Action History hiển thị lịch sử các lệnh điều khiển LED, gồm thiết bị, thao tác, trạng thái và thời gian ghi nhận. Người dùng có thể tìm kiếm theo thời gian, lọc theo LED, thao tác ON/OFF và trạng thái PENDING/SUCCESS/FAILED. Các điều kiện được kết hợp để thu hẹp kết quả, đồng thời hỗ trợ sắp xếp và phân trang.



Hình 15 Giao diện Action History

4.6 Giao diện Profile

Trang Profile hiển thị thông tin cá nhân và các liên kết đến GitHub, Figma, tài liệu API và báo cáo PDF.



Hình 16 Giao diện Profile

KẾT LUẬN

Tổng kết

Đề tài "Thiết kế và xây dựng hệ thống IoT giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển đèn LED từ xa" đã xây dựng mô hình kết nối phần cứng ESP32 với máy chủ và giao diện web. Hệ thống thực hiện thu thập, lưu trữ và hiển thị dữ liệu cảm biến, đồng thời cho phép người dùng điều khiển độc lập ba LED.

Các chức năng đã triển khai gồm đăng nhập, hiển thị Dashboard và biểu đồ, tra cứu dữ liệu cảm biến, điều khiển LED, xem lịch sử thao tác và thông tin Profile. Các công cụ tìm kiếm, lọc, sắp xếp và phân trang hỗ trợ người dùng tra cứu dữ liệu. Quá trình điều khiển có phản hồi và giới hạn thời gian chờ, giúp phân biệt yêu cầu đã tiếp nhận với lệnh đã thực hiện thành công.

Quá trình thực hiện đề tài giúp vận dụng kiến thức về lập trình nhúng, truyền thông MQTT, cơ sở dữ liệu và phát triển ứng dụng web. Tuy nhiên, mô hình hiện chỉ sử dụng một ESP32 và ba LED. Dữ liệu ánh sáng chưa được hiệu chuẩn thành độ rọi thực tế; độ chính xác cảm biến và khả năng vận hành liên tục chưa được đánh giá đầy đủ. Hệ thống cũng chưa có chức năng cảnh báo và điều khiển tự động.

Hướng phát triển

Trong thời gian tiếp theo, đề tài có thể được phát triển theo các hướng sau:

- Hiệu chuẩn cảm biến và đối chiếu dữ liệu với thiết bị đo chuẩn.
- Bổ sung cảnh báo và điều khiển tự động theo ngưỡng môi trường.
- Mở rộng quản lý nhiều ESP32 và phân quyền theo thiết bị.
- Tăng cường bảo mật, triển khai máy chủ để truy cập qua Internet.
- Mở rộng điều khiển thiết bị thực tế bằng mạch chấp hành và bảo vệ phù hợp.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, “Arduino ESP32 Documentation.”

[2] MQTT.org, “MQTT: The Standard for IoT Messaging.”

[3] Eclipse Mosquitto, “Documentation.”

[4] Express.js, “Express — Node.js Web Application Framework.”

[5] Oracle, “MySQL 8.0 Reference Manual.”

[6] Socket.IO, “Introduction.”

[7] Chart.js, “Chart.js Documentation.”

[8] MDN Web Docs, “Fetch API.”